

Herencia

Clase 5

Repaso POO

- Un objeto es una **instancia de su clase**
- Una clase representa un nuevo tipo de dato más representativo al problema que estamos resolviendo.
- Las clases y los objetos tienen **atributos y métodos** (mensajes).
- Existen atributos y métodos privados que solo se deberían acceder desde el mismo objeto que los contiene, al menos que el programador decida lo contrario.
- Métodos especiales: `__init__` , `__str__` , `__eq__` ...

Herencia

- Es uno de los conceptos más importantes de la POO.
- La herencia permite que una clase pueda heredar los atributos y métodos de otra clase, que se “agregan” a los propios.
- Este concepto permite “**extender**” una clase.
- La clase que hereda se denomina clase derivada (hija, **subclase**) y la clase de la cual se deriva se denomina clase **base** (padre).
- La herencia permite la **reutilización de código** para evitar la duplicación de código y promueve la eficiencia y la consistencia en el desarrollo de software.

Ejemplo

class Empleado:

```
def __init__(self, nombre, dni, salario):  
    self.nombre = nombre  
    self.salario = salario  
    self.dni = dni
```

```
def trabaja(self):  
    print("Estoy trabajando como empleado")
```

```
def __str__(self):  
    return f"Nombre: {self.nombre}"
```

class NoDocente(**Empleado**):

```
def __init__(self, nombre, dni, cargo, salario=1000):  
    super().__init__(nombre, dni, salario)  
    self.cargo = cargo
```

class Docente(**Empleado**):

```
def __init__(self, nombre, dni, materia, salario=1500):  
    super().__init__(nombre, dni, salario)  
    self.materia = materia
```

```
def trabaja(self):  
    print("Estoy trabajando como Docente")
```

Herencia

- Una clase hija **hereda** los atributos y métodos de la clase padre.
- Una clase hija **puede extender** (agregar) sus propios atributos y métodos.
- Una clase hija puede **sobreescribir** los atributos y métodos de su clase base.
- Facilita el mantenimiento al permitir cambios en una clase base que se reflejan en todas sus subclases.

Ejemplo

```
class Padre:
```

```
    def __init__(self):
```

```
        self.valor = 5
```

```
        self.base = 10
```

```
    def funcion(self):
```

```
        print("funcion Padre")
```

```
    def __str__(self):
```

```
        return "Padre"
```

```
class Hijo(Padre):
```

```
    def __init__(self):
```

```
        #super().__init__()
```

```
        self.valor = 7 + self.valor
```

```
    def funcion(self):
```

```
        print("funcion Hijo")
```

```
    def __str__(self):
```

```
        return super().__str__() + "Hijo"
```

```
if __name__ == '__main__':
```

```
    p = Padre()
```

```
    print(p.valor)
```

```
    p.funcion()
```

```
    print(p)
```

```
    print()
```

```
    h = Hijo()
```

```
    print(h.valor)
```

```
    h.funcion()
```

```
    print(h)
```

```
    print()
```

```
    print("Es instancia de Padre" if isinstance(h, Padre) else "NO  
es instancia de Padre")
```

```
    print("Es subclase de Padre" if issubclass(Hijo, Padre) else  
"NO es subclase de Padre")
```

Ejercicio 1

Sistema de Animales: Crea la clase base *Animal* con atributos y métodos comunes a todos los animales, como nombre, edad y `emitir_sonido()`. Luego, crea subclases como *Perro*, *Gato* y *Pájaro* que hereden de la clase *Animal* y proporcionen implementaciones específicas para el método `emitir_sonido()`.

Ejercicio 2

Se cuenta con una clase CuentaBancaria de la cual se deben crear 2 clases derivadas. Una para las cuentas de débito y otra para las de crédito. Ambas deben ser creadas con un saldo de 0 y 100000 pesos respectivamente. Se deben implementar los métodos de retirar y depositar montos solo si la contraseña es correcta y si cuenta con el saldo suficiente.

Por cada retiro las cuentas de débito tienen un recargo de 10 pesos. Las cuentas de crédito tienen un límite, solo se puede retirar como máximo 10000 pesos por cada retiro.